Київський національний університет імені Тараса Шевченка
Факультет інформаційних технологій
Кафедра програмних систем і технологій

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: УДК] На правах рукопису

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: ТОЧНА ОФІЦІЙНА НАЗВА ВИДУ ДОКУМЕНТА]

Тема:

«Розробка архітектури АІНС на основі великих мовних моделей для реалізації діалогового агента з функціями репетитора»

Спеціальність - 121 «Інженерія програмного забезпечення»

ПОЯСНЮВАЛЬНА ЗАПИСКА

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: ПІБ студента, номер групи]

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: ПІБ наукового керівника, науковий ступінь, посада]

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: кафедральні реквізити, підписи, службові дати]

Київ - 2026

СЛУЖБОВІ СТОРІНКИ / ЗАВДАННЯ

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: перевірити, чи обов'язкові для магістерської курсової роботи сторінки «Завдання», «Календарний план» та інші службові форми; за потреби оформити їх за кафедральним шаблоном].

У поточній автоматично згенерованій чернетці ці сторінки навмисно не деталізовано, оскільки реквізити затвердження, дати, підписи, протоколи кафедри та інші службові поля не можуть бути достовірно визначені без ручного заповнення.

АНОТАЦІЯ

Магістерська курсова робота: [ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: додати кількість сторінок, рисунків, таблиць, додатків, джерел].

Тема роботи: розробка архітектури адаптивної інтелектуальної навчальної системи на основі великих мовних моделей для реалізації діалогового агента з функціями репетитора.

Об'єкт дослідження - процес адаптивного навчання користувача за допомогою діалогового агента в межах інтелектуальної навчальної системи.

Предмет дослідження - архітектурні моделі, методи адаптації великих мовних моделей, механізми пам'яті агента та програмні засоби реалізації vertical slice для навчання базового JavaScript.

Мета роботи - спроєктувати узгоджену архітектуру AIHC із LLM-ядром і продемонструвати її працездатність на мінімальному вертикальному сценарії: генерація вправи, написання розв'язку, запуск тестів та надання адаптивного фідбеку.

У роботі систематизовано класичні компоненти ITS, проаналізовано сучасні підходи до використання LLM в освіті, запропоновано архітектуру агента-репетитора з моделлю стану студента, матрицею рішень і багаторівневою організацією пам'яті, а також реалізовано мінімальний browser-based прототип із локальним LLM runtime, Web Worker sandbox та fallback-логікою.

Реальні результати педагогічного експерименту у поточній версії відсутні; замість цього подано чесно оформлений дизайн експерименту та план оцінювання для подальшого дослідження.

Ключові слова: AIHC, ITS, великий мовний модель, LLM, діалоговий агент, репетитор, Ollama, Llama, JavaScript, адаптивний фідбек.

ABSTRACT

Master's coursework project: [MANUAL INPUT REQUIRED: add the final number of pages, figures, tables, appendices, and references].

The study focuses on the architecture of an adaptive intelligent tutoring system built around large language models for a dialogue-based tutor agent.

The object of study is adaptive tutoring mediated by a conversational agent. The subject of study includes architectural components, memory organization, decision policies, and implementation techniques for a JavaScript tutoring prototype.

The goal is to design an architecture-first AINS model and support it with a minimal yet working vertical slice that covers exercise generation, student code editing, client-side test execution, and adaptive feedback.

The document separates conceptual architecture from practical implementation. A local Ollama-based runtime, Web Worker sandboxing, JSON retry and fallback logic, and an honest experimental design without fabricated educational outcomes are used as core principles.

Keywords: adaptive intelligent tutoring system, large language model, dialogue agent, tutoring policy, JavaScript education, Ollama, local LLM.

ЗМІСТ

Оновіть поле змісту у Word після відкриття документа.

СПИСОК УМОВНИХ ПОЗНАЧЕНЬ

AIHC - адаптивна інтелектуальна навчальна система

ITS - Intelligent Tutoring System

LLM - Large Language Model

RAG - Retrieval-Augmented Generation

JSON - JavaScript Object Notation

API - Application Programming Interface

UI - User Interface

JS - JavaScript

Web Worker - ізольований потік браузера для клієнтського виконання коду

UserState - спрощена модель стану студента в прототипі

ВСТУП

Розвиток адаптивних інтелектуальних навчальних систем знову став одним із центральних напрямів освітніх технологій після появи великих мовних моделей. Якщо класичні ITS орієнтувалися переважно на явно задані правила, предметні моделі та обмежені діалогові інтерфейси, то LLM відкрили можливість будувати природніший діалог, оперативно генерувати навчальний контент і адаптувати зворотний зв'язок до поведінки конкретного студента [1], [6], [7], [8].

Для освітніх застосувань цього недостатньо розглядати лише якість текстової генерації. Ключовим стає архітектурне питання: які саме компоненти має містити система, як вони взаємодіють, де формується модель студента, як організовується пам'ять агента, яким чином ухвалюється педагогічна дія та як забезпечується безпечне виконання інструментальних операцій [2], [3], [9], [12].

Предметною областю цієї курсової роботи обрано навчання базового JavaScript. Такий вибір зручний з методичної та технічної точок зору: для невеликих програмістських вправ легко сформулювати тестовані критерії коректності, а процес взаємодії студента з репетитором природно вкладається у цикл «згенерувати задачу -> написати розв'язок -> запустити тести -> отримати адаптивний фідбек» [14], [15], [16], [17], [18].

Метою роботи є проєктування архітектури AIHC на основі великих мовних моделей для реалізації діалогового агента з функціями репетитора та демонстрація цієї архітектури на мінімальному, але реальному vertical slice прототипу.

Для досягнення поставленої мети необхідно розв'язати такі завдання:

1. проаналізувати класичні підходи до побудови ITS/AIHC та виділити їхні обов'язкові архітектурні компоненти;
2. узагальнити можливості й обмеження великих мовних моделей у навчальних сценаріях;
3. сформулювати функціональні та нефункціональні вимоги до системи;

4. запроєктувати модель студента, модель знань, стратегію репетитора, потоки даних і організацію пам'яті агента;
5. реалізувати мінімальний browser-based прототип для курсового vertical slice з локальним LLM runtime та безпечним тестовим запуском коду;
6. запропонувати дизайн експериментальної перевірки без вигадування педагогічних результатів.

Об'єкт дослідження - процес адаптивної навчальної взаємодії між студентом і діалоговим агентом у межах інтелектуальної навчальної системи.

Предмет дослідження - архітектурні моделі, механізми пам'яті, політики прийняття рішень і програмні засоби побудови AIHC на основі LLM для сценарію tutor-assisted вивчення JavaScript.

У роботі використано методи системного аналізу, порівняльного огляду літератури, модульного проєктування програмних систем, прототипування, тестової функціональної валідації та формального опису агентної поведінки.

Наукова новизна роботи має стриманий характер і полягає не у створенні нової моделі навчання як такої, а у синтезі архітектурного підходу до AIHC, де LLM розглядається як один із компонентів ширшої системи з явною моделлю стану студента, політикою педагогічних дій, розмежуванням типів пам'яті та безпечним механізмом виконання програмних вправ.

Практична цінність полягає у створенні прототипу, який може бути розширений у повну магістерську кваліфікаційну роботу: додаванням довготривалої пам'яті, RAG-шару, реального контентного сховища, інструментів аналітики та повноцінного педагогічного експерименту.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ АДАПТИВНИХ ІНТЕЛЕКТУАЛЬНИХ НАВЧАЛЬНИХ СИСТЕМ

1.1. Поняття та класифікація інтелектуальних навчальних систем (ITS/AIHC)

Інтелектуальна навчальна система традиційно розглядається як програмно-методичний комплекс, який намагається наблизити індивідуальне навчання до роботи з людським викладачем. На відміну від статичних e-learning платформ, ITS адаптує зміст, складність завдань, спосіб пояснення та форму зворотного зв'язку відповідно до поточного стану студента [1], [5], [6], [28].

Термін AIHC у межах цієї роботи використовується як сучасніше позначення тієї самої логіки, але з акцентом на адаптивність, агентність і поєднання класичних освітніх моделей з генеративними компонентами. Важливо, що навіть у LLM-орієнтованих системах адаптивність не виникає автоматично з самої моделі; вона вимагає явних архітектурних механізмів збору спостережень, зберігання стану та вибору педагогічних дій.

1.2. Архітектурні компоненти AIHC

Класична архітектура ITS спирається принаймні на чотири логічні підсистеми: модель предметної галузі, модель студента, педагогічну модель і інтерфейс взаємодії. У сучасних роботах цей поділ уточнюється, але не зникає: саме він дозволяє відокремити знання про предмет, знання про учня та знання про педагогічну стратегію [1], [2], [3], [4], [28].

Для цієї курсової роботи така декомпозиція принципова. Якщо одразу звести систему до одного LLM-виклику, втрачається можливість пояснити, де саме формується адаптивність. Тому в подальших розділах LLM розглядатиметься як генеративний шар, але не як повний замінник студентської моделі, контентної моделі та політики репетитора.

1.3. Діалогові агенти у навчальних системах

Натуральномовний діалог є природним інтерфейсом для навчання, оскільки дозволяє ставити уточнювальні запитання, пояснювати помилки та підтримувати мотивацію. Ранні системи з діалоговими можливостями були обмежені жорсткими сценаріями, тоді як нові LLM-агенти забезпечують значно вищу мовну гнучкість [14], [20], [22].

Особливо перспективним цей підхід є для навчання програмування. Тут агент може поєднати генерацію задач, аналіз спроб розв'язку, інтерпретацію результатів тестів і формування підказок без негайного розкриття повного коду, що узгоджується з підходами сучасних досліджень у programming education [14], [15], [16], [17], [18], [26], [27].

1.4. Огляд сучасних систем-репетиторів на основі ШІ

Сучасний ландшафт AI tutor systems містить як комерційні продукти, так і дослідницькі системи. Продуктові системи демонструють життєздатність формату, але зазвичай мало розкривають внутрішню архітектуру; академічні праці, навпаки, частіше дають детальний опис модулів і методів оцінювання [6], [7], [8], [19], [21], [23], [24].

Система / напрям	**Фокус**	**Механізм адаптації**	**Релевантність для цієї роботи**
Khanmigo	Підтримка навчання та викладання	Діалоговий AI tutor і teacher assistant	Підходить як приклад продуктового AI tutor [23]
Duolingo Max	Мовне навчання	Roleplay, Explain My Answer, GPT-4-based UX	Корисний для аналізу стилю фідбеку [24]
ITS-CAL	Навчання програмування	Структуроване адаптивне середовище для coding tasks	Дає приклад доменної адаптації [20]
LLM feedback studies	Програмування	Генерація коментарів до коду і	Показують сильні й слабкі

		педагогічних підказок	сторони LLM у CS education [15], [17], [18], [26]

Для курсової роботи важливо не копіювати жодну з цих систем буквально, а виділити архітектурний інваріант: сучасний AI tutor має поєднувати мовну генерацію з контролем допомоги, явним поданням стану студента та механізмом перевірки зовнішніх дій.

1.5. Висновки до розділу 1

Аналіз предметної галузі показав, що архітектура AIHC не може зводитися до одного мовного модуля. Необхідними залишаються студентська модель, контентна модель, педагогічна політика, інтерфейс взаємодії та механізм спостереження за діями студента. Для сценарію навчання JavaScript найбільш логічною є архітектура, у якій LLM поєднується з інструментальним запуском тестів і адаптивною політикою репетитора [1], [2], [14], [15], [20], [22].

РОЗДІЛ 2. ВЕЛИКІ МОВНІ МОДЕЛІ ЯК ОСНОВА ДІАЛОГОВОГО АГЕНТА

2.1. Архітектура трансформерів та принципи роботи LLM

Великі мовні моделі базуються на архітектурі трансформера та навчаються передбачати наступні токени на великих корпусах тексту. На практиці це дає універсальний механізм, який можна використовувати для пояснення, формування підказок, перефразування й частково для планування дій, якщо система обмежує формат відповіді та подає моделі структурований контекст [7], [8].

У навчальних застосуваннях важливо відділяти дві ролі LLM: мовну й керувальну. Мовна роль полягає у створенні зрозумілих відповідей, а керувальна - у виборі, що саме треба сказати в даному педагогічному контексті. Друга роль не повинна повністю покладатися на статистичну генерацію; вона потребує додаткової політики та пам'яті системи.

2.2. Порівняння LLM для навчальних застосунків

Для освітніх систем доступні як закриті frontier-моделі, так і відкриті локальні моделі. Закриті сервіси зазвичай демонструють вищу якість міркування й пояснень, однак створюють залежність від зовнішнього API, підвищують ризики для приватності й ускладнюють відтворюваність. Локальні open-source моделі є слабшими в середньому, але дають більше контролю над витратами, обробкою даних і правилами інтеграції [7], [8], [10], [17], [18], [19], [29].

Для задачі базового JavaScript tutor не потрібна максимальна генеративна потужність, натомість важливі локальність, передбачуваність і можливість працювати з невеликим структурованим JSON-контрактом. Саме тому в роботі обрано локальний runtime `Ollama + llama3.1:8b` як цільове рішення для прототипу [25].

2.3. Prompt engineering, RAG, fine-tuning як методи адаптації

На практиці адаптація LLM до навчального домену може здійснюватися щонайменше трьома способами: через prompt engineering, через retrieval-augmented

generation та через fine-tuning. Prompt engineering є найдоступнішим шляхом для курсового проєкту, оскільки дозволяє нав'язати формат JSON, заборону на повне розкриття розв'язку і стиль репетитора без окремого етапу донавчання [11], [12], [25].

RAG є логічним наступним кроком для розвитку системи, адже дозволяє під'єднати зовнішнє сховище навчального контенту, прикладів і теоретичних пояснень. Fine-tuning у межах поточної курсової не реалізується через обмеження масштабу, але в архітектурному описі він розглядається як потенційний засіб стабілізації стилю та якості педагогічного фідбеку.

2.4. Обмеження LLM: галюцинації, пам'ять, етика, приватність

Освітнє застосування LLM пов'язане з ризиками галюцинацій, надмірної допомоги, відсутності стабільної довготривалої пам'яті та з етичними питаннями щодо прозорості, приватності й залежності студента від системи [9], [10], [13], [29]. Для репетиторського сценарію ці ризики є особливо критичними: неправильно сформульована підказка може не просто помилково інформувати, а й зруйнувати навчальний ефект завдяки передчасному розкриттю відповіді.

Звідси випливають архітектурні наслідки для курсової роботи: необхідність контролю формату відповіді, retry/fallback при невалідному JSON, окремого механізму політики репетитора, обмеження на розкриття розв'язку та безпечного виконання лише тих інструментальних дій, які не вимагають server-side eval.

2.5. Обґрунтування вибору локальної open-source Llama для прототипу

Вибір локальної Llama через Ollama зумовлено кількома факторами. По-перше, це відповідає вимозі використовувати безкоштовний локальний runtime без звернення до платних хмарних API. По-друге, Ollama надає достатньо простий HTTP API для інтеграції у lightweight-прототип. По-третє, модель класу 8B достатня для генерації невеликих задач, коротких підказок та структурованих відповідей у межах базового навчального сценарію [17], [18], [19], [25].

Водночас у роботі прямо визнається, що такий вибір є компромісом: локальна модель може поступатися сильнішим закритим системам за точністю та стабільністю. Саме тому в архітектурі одразу передбачено нормалізацію відповіді, fallback-вправи та шаблонний фідбек на випадок, коли LLM недоступна або повертає некоректні дані.

Отже, LLM у межах цієї роботи виступає не як автономний педагог, а як гнучкий генеративний компонент, вбудований у ширшу архітектуру AIHC з явною моделлю стану, механізмом пам'яті та політикою ухвалення рішень [7], [9], [12], [25].

РОЗДІЛ 3. ПРОЄКТУВАННЯ АРХІТЕКТУРИ АІНС

3.1. Функціональні та нефункціональні вимоги

Система проектується не як універсальний освітній портал, а як вертикальний сценарій для tutor-assisted вивчення базового JavaScript. Це дозволяє сформулювати чіткі вимоги й водночас залишити місце для подальшого масштабування.

Код	Функціональна вимога	Статус у прототипі
FR1	Згенерувати невелику вправу з JavaScript у структурованому форматі	реалізовано
FR2	Дозволити студенту написати або відредагувати розв'язок	реалізовано
FR3	Запустити тести лише на клієнті, у Web Worker з таймаутом	реалізовано
FR4	Надати адаптивний фідбек без негайного повного розв'язку	реалізовано
FR5	Підтримати мінімальну модель стану студента в межах сесії	реалізовано
FR6	Обробити невалідний JSON від моделі через retry/fallback	реалізовано

Код	Нефункціональна вимога	Примітка
NFR1	Використовувати локальний open-source runtime	цільовий runtime: Ollama + llama3.1:8b
NFR2	Не виконувати user code	виконано

	на сервері	
NFR3	Забезпечити модульну архітектуру з явним поділом відповідальностей	виконано
NFR4	Забезпечити пояснювану логіку педагогічного рішення	виконано через TutorPolicy
NFR5	Мати деградаційний режим без LLM	виконано
NFR6	Не вигадувати результати експериментів	виконано в документі

3.2. Загальна архітектура системи

Архітектура пропонованої AIHC складається з п'яти логічних шарів: інтерфейсного, шару сприйняття та інтерпретації запиту, шару педагогічного міркування, шару інструментальних дій і шару пам'яті. Центральним елементом виступає агент-репетитор, який не просто генерує текст, а працює в циклі спостереження, оновлення стану й вибору наступної дії [1], [11], [12].

На практичному рівні прототип реалізує лише частину цієї архітектури, але вона вже достатня, щоб продемонструвати головний принцип: генерація вправи й текстовий фідбек мають бути відокремлені від виконання студентського коду та від моделі стану користувача. Таке розділення зменшує ризик змішування педагогічної логіки та інструментального рантайму.

Компонент	Призначення	Статус
Chat UI	Інтерфейс студента, показ вправи, коду, результатів тестів і фідбеку	реалізовано
Perception Module	Класифікація наміру й нормалізація запиту	описано концептуально
SessionMemory	Поточний стан, transcript	реалізовано

	tail, історія помилок	
Summary Memory	Стисле зведення попередніх сесій	описано концептуально
Vector/Profile Memory	Довготривалий профіль студента між сесіями	описано концептуально
Knowledge Memory	Контентна база, RAG, концепти, пояснення	описано концептуально
TutorPolicy / Reasoning Module	Вибір педагогічної дії	реалізовано у rule-based вигляді
ExerciseGenerator	Створення вправи через LLM або fallback	реалізовано
ClientTestRunner	Запуск тестів у Web Worker	реалізовано
FeedbackEvaluator	Формування адаптивного фідбеку	реалізовано
Generation Module / LLM	Генерація структурованих exercise/feedback JSON	реалізовано частково через Ollama API

[Плейсхолдер для рисунка]

Рисунок 3.1 - Загальна компонентна архітектура AIHC для JavaScript tutor.

Текстові джерела для діаграм винесено в окремий файл `docs/codex/DIAGRAM_SOURCES.md`, що дозволяє надалі замінити плейсхолдери на відрендерені схеми без зміни структури документа.

3.3. Модель студента

У центрі персоналізації перебуває модель студента. Повноцінні ITS використовують для цього складніші probabilistic або knowledge tracing механізми, але для vertical slice доцільно застосувати спрощений вектор стану `UserState`, який усе ж зберігає ключові сигнали для педагогічного рішення [2], [3], [4].

Поле UserState	Призначення

knowledgeLevel	Груба оцінка поточного рівня знань студента
errorHistory	Список останніх помилок або типів збоїв
confidence	Суб'єктивно-операційна оцінка впевненості системи у стані студента
currentTopic	Поточна навчальна тема
attemptsCount	Кількість спроб для поточної вправи
lastAction	Остання педагогічна дія агента

Оновлення `UserState` відбувається після кожної значущої взаємодії. Результати тестів, тип першої помилки, повторення того самого збою та запит на пояснення інтерпретуються як сигнали для оновлення впевненості, історії помилок і вибору наступної дії. У повній системі до цього вектора могли б додаватися concept-level mastery, preferred explanation style та довготривалий профіль.

[Плейсхолдер для рисунка]

Рисунок 3.2 - Структура UserState та її зв'язок із SessionMemory.

3.4. Модель знань і навчального контенту

Модель знань у повній AIHC повинна поєднувати концепти предметної галузі, навчальні цілі, типові помилки та набір контентних артефактів: вправи, тести, пояснення, контрольні запитання. У класичному ITS це часто реалізується через експліцитну предметну модель, а в сучасних LLM-based системах може бути доповнене retrieval-шаром та векторним індексом [3], [11].

У прототипі ця модель навмисно спрощена до JSON-репрезентації вправи з полями `title`, `prompt`, `starterCode`, `functionName`, `tests`, `concepts`, `rubric`. Таке рішення дозволяє зберегти місце для майбутнього RAG, але не розширювати практичну частину понад необхідний мінімум.

Архітектурно knowledge memory поділяється на чотири типи сутностей: концепти JavaScript, шаблони задач, перевірочні тести та пояснювальні матеріали. У повній версії системи ці сутності можуть бути пов'язані у вигляді графа концептів і retrieval-індексу; в поточній версії їх роль частково виконує вбудований fallback-набір вправ і prompt-контракт для LLM.

3.5. Модель і стратегії репетитора

3.5.1. Формальне визначення агента-репетитора

Діалоговий агент у цій роботі визначається як керована система, що підтримує навчальний діалог і виконує педагогічно осмислені дії, спираючись на спостереження за студентом. На відміну від звичайного чат-бота, агент-репетитор пов'язує мовну відповідь із контекстом навчальної задачі, поточним станом користувача та інструментальними результатами перевірки розв'язку.

```
S = K × E × C × Topic × Attempts × M
A = {generate_exercise, give_minimal_hint, give_targeted_hint,
     explain_concept, celebrate_and_reflect, request_retry, escalate}
T: S × O -> S
P: S × O -> A
R: S × A × Ctx -> Text
```

Тут `K` позначає оцінку знань, `E` - історію помилок, `C` - впевненість, `Topic` - поточну тему, `Attempts` - кількість спроб, `M` - пам'ять сесії, `O` - нові спостереження, `A` - множину дій агента, `T` - функцію переходу стану, `P` - політику, а `R` - функцію генерації відповіді. Такий опис дозволяє відокремити міркування агента від суто мовної генерації {cite('S11', 'S12')}.

3.5.2. Архітектура агента - компонентна модель

Компонентна модель агента складається з таких блоків. `Perception Module` інтерпретує вхідний запит і визначає, чи студент просить пояснення, чи надсилає нову спробу. `Memory Module` акумулює поточний контекст і довготривалі відомості. `Reasoning Module` оцінює стан студента та обирає педагогічну дію. `Action Module` викликає генерацію вправи, перевірку тестів або пояснення. `Generation Module` формує кінцеву текстову відповідь на основі політики та контексту.

У прототипі ця декомпозиція реалізована не повністю: роль `Perception Module` виконують прості евристики розпізнавання пояснювального запиту, а роль `Generation Module` виконує локальна LLM або fallback-механізм. Однак саме така схема буде використана як основа повнішої системи в магістерському проєкті.

3.5.3. Матриця рішень і minimal feedback strategy

Ключова вимога до агента-репетитора полягає в тому, щоб допомога була мінімально достатньою. Система не повинна відразу видавати готовий код, якщо студент ще може просунутися самостійно. Тому політика будується за принципом ескалації підказок залежно від кількості спроб та повторюваності помилки.

Стан користувача	Дія агента	Тип відповіді
Усі тести пройдено	celebrate_and_reflect	Підкріпити успіх і запропонувати коротку рефлексію
Перша помилка	minimal_hint	Натякнути на ідею без готового розв'язку
Повторна помилка	targeted_hint	Вказати на конкретний збій і пов'язаний концепт
Синтаксична помилка	syntax_repair	Підказати зону пошуку помилки
Запит на пояснення	concept_explanation	Дати коротке концептуальне пояснення без повного коду

У коді ця матриця реалізована в модулі `TutorPolicy` через rule-based правила. Для більш складної системи вона може бути замінена комбінацією rule engine, bandit-стратегії або policy learning, але для курсової роботи важливо насамперед показати сам принцип явного розділення педагогічної дії та генеративної реалізації.

3.5.4. Типові сценарії взаємодії

- Правильна відповідь: агент фіксує проходження тестів, підсилює впевненість студента та пропонує коротке пояснення власними словами.
- Помилка: агент дає мінімальний натяк, орієнтований на найперший збій або простий приклад.
- Повторна помилка: агент переходить до targeted hint і конкретніше називає, який фрагмент логіки слід перевірити.
- Запит на пояснення: агент переключається в режим concept explanation і дає теоретичне пояснення без завершеного розв'язку.

3.5.5. Цикл роботи агента: Reason -> Act -> Observe

Робочий цикл агента описується патерном `Reason -> Act -> Observe`. На етапі `Reason` система інтерпретує нові спостереження й оновлює `UserState`. На етапі `Act` обирається педагогічна дія: створити вправу, дати підказку, пояснити концепт або запросити нову спробу. На етапі `Observe` система отримує нові результати, зокрема outcome тестів, і повторює цикл [12].

[Плейсхолдер для рисунка]

Рисунок 3.3 - Цикл роботи агента за патерном Reason -> Act -> Observe.

3.5.6. Організація пам'яті агента

Тип пам'яті	Призначення	Статус
Buffer / Session Memory	Контекст поточної сесії, останні повідомлення, поточна вправа, результат тестів	реалізовано
Summary Memory	Стислий підсумок попереднього діалогу, придатний для наступної сесії	концептуально
Vector / Profile Memory	Стійкий профіль студента між сесіями, embeddings і патерни	концептуально

	труднощів	
Knowledge Memory	Навчальний контент, концепти, приклади, RAG-корпус	концептуально

Такий поділ потрібен для того, щоб не змішувати короткочасний контекст із довготривалою персоналізацією та із зовнішнім предметним знанням. Саме це є однією з ключових архітектурних тез роботи: пам'ять агента не повинна ототожнюватися з контекстним вікном LLM.

3.5.7. Роль LLM і її обмеження

У пропонованій AIHC LLM виконує роль ядра модуля генерації, але не є єдиним носієм логіки системи. Вона перетворює структурований контекст у зрозумілу підказку або вправу, однак сама політика, модель стану та безпечний execution runtime мають бути винесені за межі моделі. Такий підхід дозволяє зменшити вплив галюцинацій і краще контролювати освітню доцільність відповіді [9], [10], [13].

Таким чином, найбільш цінним результатом цього розділу є не конкретний інструмент або промпт, а архітектурна схема, яка пояснює місце кожного модуля в системі та відразу розділяє концептуальний рівень AIHC і той обмежений набір компонентів, який реально реалізовано в прототипі.

РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ЕКСПЕРИМЕНТАЛЬНА ПЕРЕВІРКА

4.1. Вибір технологічного стеку

Оскільки в репозиторії на початку роботи не було прикладної реалізації, практичну частину свідомо обмежено до одного мінімального vertical slice. Метою було не створити повну платформу, а отримати реальний кодовий артефакт, що демонструє архітектурні рішення з розділу 3.

Шар	Технологія	Обґрунтування
Frontend	HTML/CSS/JavaScript ES modules	мінімальний browser-based прототип без окремого фреймворку
LLM runtime	Ollama + llama3.1:8b	локальний open-source runtime відповідно до вимог роботи
Стан сесії	in-memory SessionMemory	простий і прозорий варіант для vertical slice
Запуск тестів	Web Worker + timeout	без server-side eval і з ізоляцією обчислень
Інженерні тести	node --test	перевірка pure logic модулів

У поточному середовищі локальний `ollama` binary не був встановлений, тому прототип додатково отримав деградаційний режим. Це важливий практичний висновок: для дослідницької системи потрібно з самого початку передбачати fallback-логіку, а не вважати зовнішній runtime гарантовано доступним [25].

4.2. Реалізація ключових модулів системи

У практичній частині описуються лише ті модулі, які реально присутні в коді репозиторію. Повна архітектура AIHC ширша, але в цьому підрозділі фіксується тільки фактичний обсяг реалізації.

Модуль	Файли	Функція

Chat UI	`prototype/index.html`, `prototype/src/main.js`	Форма вибору теми, редактор коду, кнопки запуску тестів, панелі стану й transcript
SessionMemory	`prototype/src/state/sessionMemory.js`	Зберігання `UserState`, поточної вправи та історії повідомлень
TutorPolicy	`prototype/src/services/tutorPolicy.js`	Rule-based матриця рішень для вибору педагогічної дії
ExerciseGenerator	`prototype/src/services/exerciseGenerator.js`	LLM JSON prompt + fallback-вправи
ClientTestRunner	`prototype/src/services/clientTestRunner.js`, `prototype/src/workers/jsSandboxWorker.js`	Клієнтський запуск тестів з таймаутом у worker
FeedbackEvaluator	`prototype/src/services/feedbackEvaluator.js`	Генерація адаптивного фідбеку через LLM або fallback
OllamaClient	`prototype/src/services/ollamaClient.js`	HTTP API, status check, retry/fallback для JSON-відповідей

Така реалізація відповідає architecture-first принципу: замість великої кількості другорядних фіч свідомо реалізовано лише ті модулі, які демонструють зв'язок між генерацією задачі, моделлю стану, тестовим запуском та адаптивним репетиторським фідбеком.

4.3. Розробка prompt-стратегій

Для прототипу сформовано дві основні prompt-стратегії: генерація вправи та формування адаптивного фідбеку. В обох випадках система вимагає лише JSON-відповідь, що спрощує валідацію й нормалізацію. Такий підхід узгоджується з

рекомендаціями щодо керованого використання LLM у навчанні та програмуванні [9], [12], [15], [16], [17], [18], [26], [27].

- стиль відповіді має бути репетиторським, коротким і підтримувальним;
- повний розв'язок не повинен видаватися одразу;
- після повторних невдалих спроб дозволяється більш конкретна, але все ще не повна підказка;
- exercise JSON повинен містити сигнатуру функції, список тестів і рубрику;
- якщо модель повертає невалідний JSON або недоступна, система переходить у fallback-режим.

Таким чином, prompt engineering у цій роботі відіграє роль засобу обмеження та структуризації генерації, а не самодостатнього механізму педагогічної адаптації.

4.4. Дизайн та проведення експерименту

4.4.1. Функціональна валідація прототипу

У поточній роботі було виконано лише ту функціональну перевірку, яку можна підтвердити реальними артефактами середовища. Освітні результати, UX-оцінки та ефективність навчання тут не вигадуються.

Об'єкт перевірки	Артефакт	Результат	Коментар
Логіка розбору JSON	`npm test`	пройдено	Перевірено екстракцію валідного JSON зі змішаного тексту
SessionMemory	`npm test`	пройдено	Перевірено reset і оновлення attemptsCount/errorHistory
TutorPolicy	`npm test`	пройдено	Перевірено explanation request, escalation і celebrate

			cases
Синтаксис JS-модулів	`node --check` по `prototype/src` і `prototype/tests`	пройдено	Помилок парсингу не виявлено
Static smoke check	`python3 -m http.server` + `curl -I /prototype/`	200 OK	Підтверджено доставку HTML-артефакту
E2E із локальною Ollama	не виконано в поточному середовищі	manual pending	Потребує встановленого Ollama і ручного запуску в браузері

Отже, інженерна функціональність vertical slice підтверджена частково: чисті модулі й базова статична доставка працюють, але повний інтерактивний сценарій з локальним Ollama має бути додатково перевірений вручну.

4.4.2. Дизайн педагогічного експерименту

Реальний педагогічний експеримент у межах цієї курсової роботи ще не проводився, тому тут подається лише чесно оформлений дизайн дослідження, який може бути використаний на наступному етапі [14], [19], [21], [22], [26], [27].

Найдоцільнішим для невеликої вибірки видається crossover-дизайн. Одна група спочатку працює з AIHC-прототипом, друга - з контрольним сценарієм без адаптивного репетиторського фідбеку; після короткого washout-періоду умови міняються місцями.

Гіпотеза	Зміст
H1	Адаптивний агент покращує post-test результат порівняно з базовим сценарієм
H2	Агент скорочує кількість безрезультатних спроб і час до коректного розв'язку
H3	Агент підвищує відчуття підтримки, не

	збільшуючи залежність від готових відповідей

Параметр	План
Цільова вибірка	24-40 студентів, які вивчають базові теми JavaScript
Процедура	pre-test -> сесія A/B -> crossover -> post-test -> delayed test
Основні метрики	правильність розв'язку, час виконання, кількість спроб, якість пояснення, відкладене запам'ятовування
Додаткові метрики	самооцінка впевненості, суб'єктивна корисність підказок
Статистичний аналіз	paired t-test або Wilcoxon; effect size через Cohen's d або Cliff's delta

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: після проведення реального експерименту додати вибірку, статистичні результати, величини ефекту та інтерпретацію.]

4.5. Аналіз результатів

На момент завершення автоматично згенерованої чернетки наявні лише результати інженерної валідації. Їх достатньо, щоб стверджувати: у репозиторії присутній реальний vertical slice, який структурно відповідає описаній архітектурі, але вони недостатні для висновків про педагогічну ефективність.

Практичний результат полягає в тому, що документ і код узгоджено. У тексті не заявляються модулі, яких немає у реалізації, а в коді реалізовано саме ті компоненти, які описано в підрозділах 4.1-4.3: `Chat UI`, `SessionMemory`, `TutorPolicy`, `ExerciseGenerator`, `ClientTestRunner`, `FeedbackEvaluator` та `OllamaClient`.

Ключове обмеження полягає в тому, що локальний Ollama runtime не був встановлений у поточному середовищі. Це означає, що частина практичної логіки

перевірялася у fallback-режимі, а повна робота з реальною локальною моделлю потребує ручного запуску.

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: додати результати A/B або crossover експерименту після збору реальних освітніх даних.]

ВИСНОВКИ

У роботі показано, що для побудови діалогового агента з функціями репетитора недостатньо розглядати лише LLM; потрібна повна архітектура AIHC із моделлю студента, моделлю знань, політикою репетитора, організацією пам'яті та механізмом інструментальних дій.

Найважливішим результатом є архітектурна схема агента, в якій виокремлено `Perception Module`, `Memory Module`, `Reasoning Module`, `Action Module` і `Generation Module`, а також формально описано стан, дії, політику й функцію переходу.

Для практичної частини реалізовано мінімальний vertical slice JavaScript tutor: генерація вправи, введення коду, запуск тестів у Web Worker, адаптивний фідбек, SessionMemory та fallback-логіка у випадку недоступності або нестабільності локальної LLM.

Документ побудовано чесно щодо емпіричної частини: реальні освітні результати не вигадувалися, а замість них подано інженерну функціональну валідацію та дизайн майбутнього педагогічного експерименту.

Отриманий прототип і текстова чернетка можуть слугувати базою для подальшого розширення в повну магістерську роботу з додаванням довготривалої пам'яті, RAG, richer content model та реального оцінювання впливу системи на навчальні результати.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

[1] Alkhatlan A., Kalita J. Intelligent Tutoring Systems: A Comprehensive Historical Survey with Recent Developments. International Journal of Computer Applications. 2019. DOI: 10.5120/ijca2019918451.

[2] Conati C., Kardan S. Student Modeling: Supporting Personalized Instruction, from Problem Solving to Exploratory Open-Ended Activities. AI Magazine. 2013. DOI: 10.1609/aimag.v34i3.2483.

[3] Abdelrahman G., Wang Q., Nunes B. P. et al. Knowledge Tracing: A Survey. ACM Computing Surveys. 2023. DOI: 10.1145/3569576.

[4] Corbett A. T., Anderson J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction. 1995. DOI: 10.1007/BF01099821.

[5] Dai C. P., Ke F. et al. Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review. Computers and Education: Artificial Intelligence. 2022. DOI: 10.1016/j.caeai.2022.100087.

[6] Letourneau K., Deslandes Martineau M., Charland P., Karran A. A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education. npj Science of Learning. 2025. DOI: 10.1038/s41539-025-00320-7.

[7] Xu H., Gan W., Qi Z., Wu J., Yu P. S. Large Language Models for Education: A Survey. arXiv preprint arXiv:2405.13001. 2024. URL: https://arxiv.org/abs/2405.13001.

[8] Wang S., Xu T., Li H., Zhang C., Liang J., Tang J. Large Language Models for Education: A Survey and Outlook. IEEE Signal Processing Magazine. 2025. DOI: 10.1109/MSP.2025.3594309.

[9] Yan L., Sha L., Zhao L., Li X. Practical and ethical challenges of large language models in education: A systematic scoping review. British Journal of Educational Technology. 2024. DOI: 10.1111/bjet.13370.

[10] Shi Y., Yu Z., Dong Y., Chen Y. Large language models in education: a systematic review of empirical applications, benefits, and challenges. Computers and Education: Artificial Intelligence. 2026. DOI: 10.1016/j.caeai.2025.100529.

[11] Lewis P., Perez E., Piktus A., Petroni F. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv preprint arXiv:2005.11401. 2020. URL: https://arxiv.org/abs/2005.11401.

[12] Yao S., Zhao J., Yu D., Du N., Shafran I., Narasimhan K. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629. 2022. URL: https://arxiv.org/abs/2210.03629.

[13] Huang L., Yu W., Ma W., Zhong W., Feng Z., Wang H. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. arXiv preprint arXiv:2311.05232. 2023. URL: https://arxiv.org/abs/2311.05232.

[14] Crow T., Luxton-Reilly A., Wuensche B. Intelligent tutoring systems for programming education. Proceedings of the 20th Australasian Computing Education Conference. 2018. DOI: 10.1145/3160489.3160492.

[15] Jacobs C., Jaschke S. Evaluating the Application of Large Language Models to Generate Feedback in Programming Education. 2024 IEEE Global Engineering Education Conference (EDUCON). 2024. DOI: 10.1109/EDUCON60312.2024.10578838.

[16] Jin I., Lee M., Shin H., Kim J. Teach AI How to Code: Using Large Language Models as Teachable Agents for Programming Education. Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024. DOI: 10.1145/3613904.3642349.

[17] Koutcheme C., Dainese N., Sarsa S., Hellas A. Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4-As-A-Judge. Proceedings of the 2024 on Innovation and Technology in Computer Science Education. 2024. DOI: 10.1145/3649217.3653612.

[18] Koutcheme C., Dainese N., Sarsa S., Hellas A. Evaluating Language Models for Generating and Judging Programming Feedback. Proceedings of the 56th ACM Technical Symposium on Computer Science Education. 2025. DOI: 10.1145/3641554.3701791.

[19] Raihan A., Siddiq F., Santos A., Zampieri M. Large Language Models in Computer Science Education: A Systematic Literature Review. Proceedings of the 56th ACM Technical Symposium on Computer Science Education. 2025. DOI: 10.1145/3641554.3701863.

[20] Lai Y.-H., Lin Y.-T. Analysis of Learning Behaviors and Outcomes for Students with Different Knowledge Levels: A Case Study of Intelligent Tutoring System for Coding and Learning (ITS-CAL). Applied Sciences. 2025. DOI: 10.3390/app15041922.

[21] LearnLM Team, Eedi, Wang A. et al. AI tutoring can safely and effectively support students: An exploratory RCT in UK classrooms. arXiv preprint arXiv:2512.23633. 2025. URL: https://arxiv.org/abs/2512.23633.

[22] El Saadawi G., Tseytlin E., Legowski E., Jukic D. A natural language intelligent tutoring system for training pathologists: implementation and evaluation. Advances in Health Sciences Education. 2008. DOI: 10.1007/s10459-007-9081-3.

[23] Khan Academy. Meet Khanmigo: Khan Academy's AI-powered teaching assistant & tutor. 2024. URL: https://www.khanmigo.ai/ (дата звернення: 14.03.2026).

[24] Duolingo. Introducing Duolingo Max, a learning experience powered by GPT-4. 2023. URL: https://blog.duolingo.com/duolingo-max/ (дата звернення: 14.03.2026).

[25] Ollama; Meta. Ollama API documentation and Llama 3.1 model page. 2024. URL: https://docs.ollama.com/api ; https://ollama.com/library/llama3.1 (дата звернення: 14.03.2026).

[26] Kiesler N., Lohr A., Keuning H. Exploring the Potential of Large Language Models to Generate Formative Programming Feedback. 2023 IEEE Frontiers in Education Conference (FIE). 2023. DOI: 10.1109/FIE58773.2023.10343457.

[27] Ma R., Shen Y., Koedinger K., Wu M. How to Teach Programming in the AI Era? Using LLMs as a Teachable Agent for Debugging. Lecture Notes in Computer Science. 2024. DOI: 10.1007/978-3-031-64302-6_19.

[28] Lin C.-C., Huang A. Y. Q., Lu O. H. T. Artificial intelligence in intelligent tutoring systems toward sustainable education: a systematic review. Smart Learning Environments. 2023. DOI: 10.1186/s40561-023-00260-y.

[29] Dong J., Bai Y., Xu Y., Zhou L. Large Language Models in Education: A Systematic Review. 2024 6th International Conference on Computer Science and Technologies in Education (CSTE). 2024. DOI: 10.1109/CSTE62025.2024.00031.

ДОДАТКИ

Додаток А. Діаграми архітектури системи

У поточній чернетці в основних розділах використано figure placeholders. Текстові джерела діаграм збережено у файлі `docs/codex/DIAGRAM_SOURCES.md` і можуть бути конвертовані в Mermaid, SVG або PNG під час фінального оформлення.

- Рисунок 3.1 - Загальна компонентна архітектура AIHC.
- Рисунок 3.2 - Структура UserState та зв'язок із SessionMemory.
- Рисунок 3.3 - Цикл роботи агента Reason -> Act -> Observe.
- Рисунок 4.1 - Runtime-схема прототипу з локальним Ollama та Web Worker.

Додаток Б. Приклади промптів

Нижче наведено скорочені фрагменти промптів, які реально використовуються в прототипі. Вони демонструють обмеження на формат відповіді та заборону на повне розкриття розв'язку.

```
Ти виконуєш роль репетитора з базового JavaScript.
Поверни лише JSON-об'єкт без пояснень.
Вправа має містити: title, prompt, starterCode, functionName,
concepts, rubric, tests.
Не включай повний розв'язок.
Ти - доброзичливий репетитор з JavaScript для початківця.
Поверни лише JSON-об'єкт.
Не давай повний розв'язок.
Врахуй attemptsCount, policy.action і перший збій у тестах.
```

Додаток В. Приклади діалогів агента

Приклад 1. Перша помилка.

```
Студент: Я запустив тести, але один сценарій не проходить.
Репетитор: Подивись на найпростіший приклад для порожнього масиву.
Репетитор: Не переписуй усе рішення, спочатку перевір, яке значення функція має
повернути в цьому випадку.
```

Приклад 2. Запит на пояснення.

```
Студент: Поясни, чому тут потрібен цикл.
Репетитор: Цикл потрібен, коли треба пройти всі елементи масиву і поступово оновлювати
результат.
Репетитор: Спробуй спершу сформулювати, яка змінна має накопичувати суму.
```

Додаток Г. Фрагменти програмного коду

Фрагмент 1. Правило ескалації підказок у `TutorPolicy`.

```
if (isExplanationRequest(studentRequest)) {
  return { action: "concept_explanation" };
}
if (runResult?.allPassed) {
  return { action: "celebrate_and_reflect" };
}
if (repeatedError || attemptsCount >= 2) {
  return { action: "targeted_hint" };
}
return { action: "minimal_hint" };
```

Фрагмент 2. Ізольоване виконання студентського коду у Web Worker.

```
const worker = new Worker(new URL("../workers/jsSandboxWorker.js", import.meta.url), {
  type: "module"
});
worker.postMessage({ userCode, functionName, tests });
```

Фрагмент 3. Оновлення SessionMemory після результату тестів.

```
this.userState.attemptsCount += 1;
this.userState.confidence = deriveConfidence(runResult);
this.userState.errorHistory = [newError, ...this.userState.errorHistory].slice(0, 8);
```